

Computer Architecture: *Memory Caches and Performance*

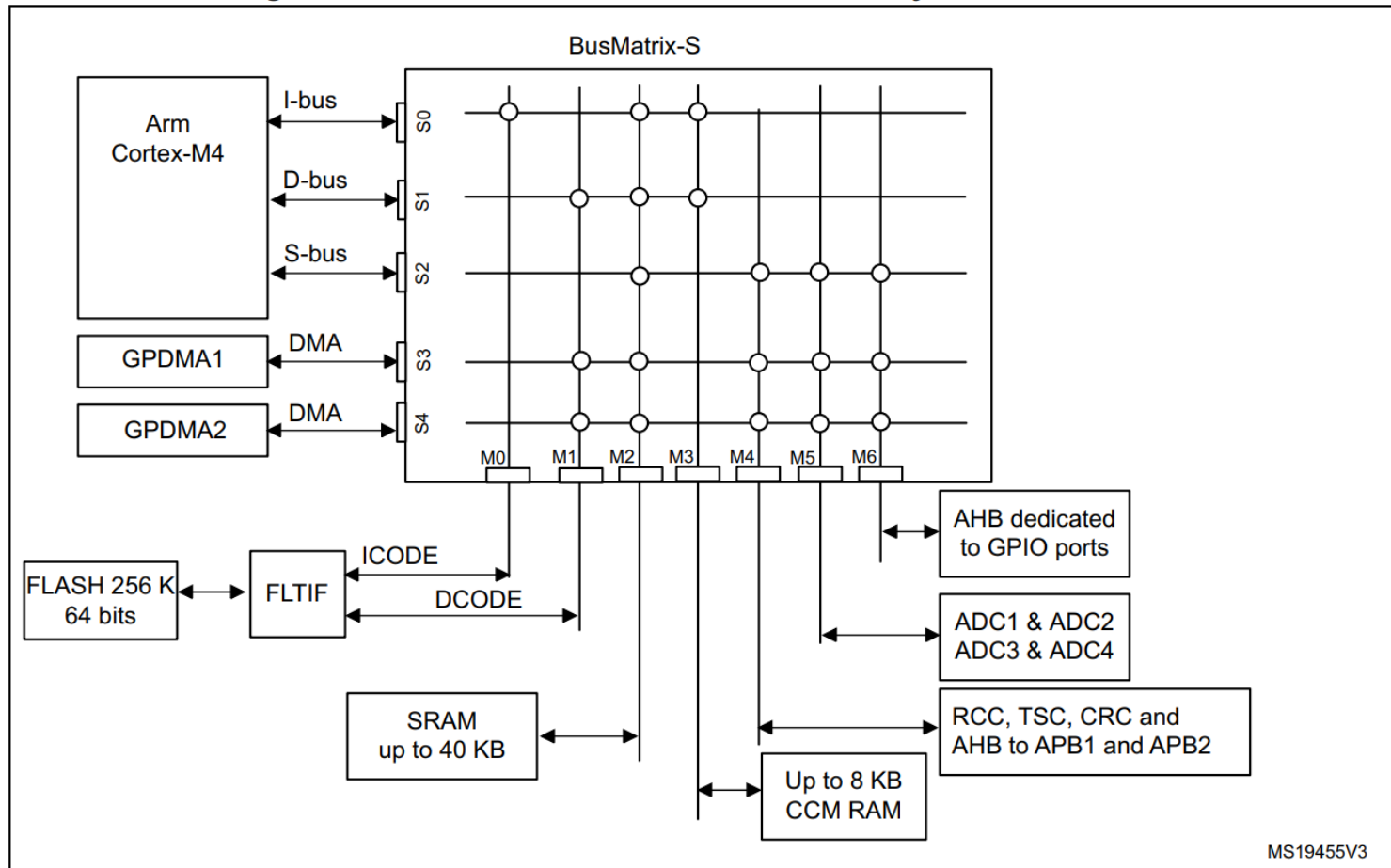
Dr. Colin O'Flynn

Lectures

- How does a program run on a machine?
- How do you compile from high-level code to machine code?
- How does the machine run? (Control Flow)
- How does the machine store data?
- How does the machine interface using buses?
- What does it mean to have “allowed” memory?
- Why is some memory faster & slower?
- **How does the machine speed up data access?**

What's all this other stuff near the CPU?

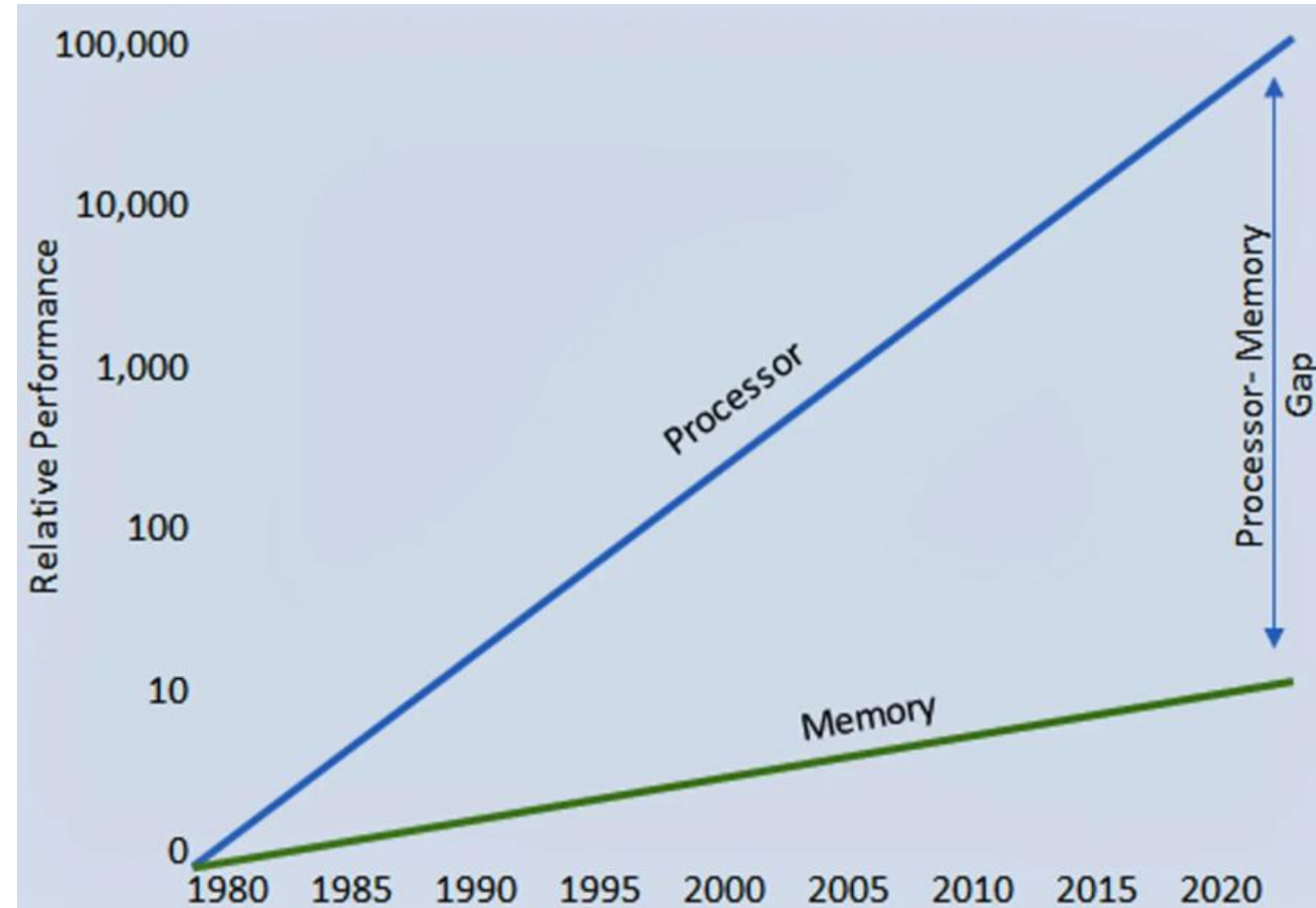
Figure 1. STM32F303xB/C and STM32F358xC system architecture



- Finally we answer how the CPU deals with latency.

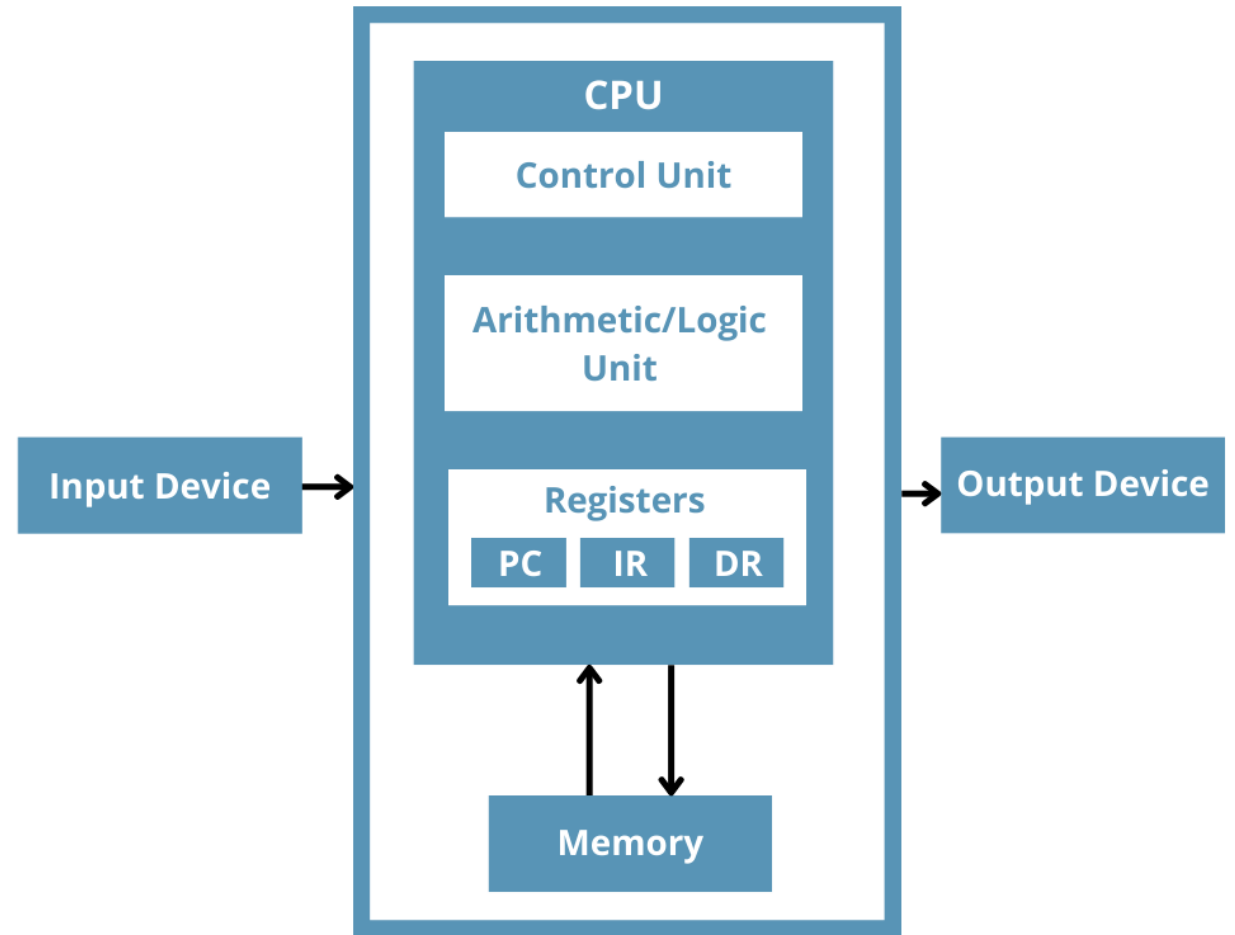
The Memory Wall

- CPU speed has increased much faster than memory speed.
- Memory becomes the limiting factor!



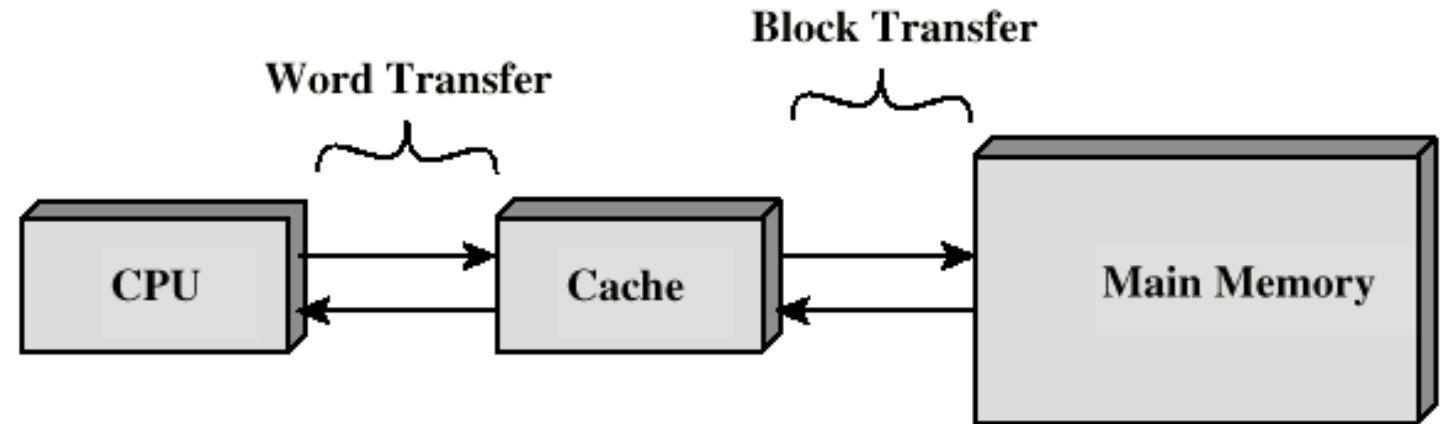
Registers → Caches

- We already have the idea of registers which are very fast...
- We know from previous lectures that registers are basically memory...
- Sooo...



Registers → Caches

- Put some memory right by the CPU!
- This memory keeps copies of data we access frequently.

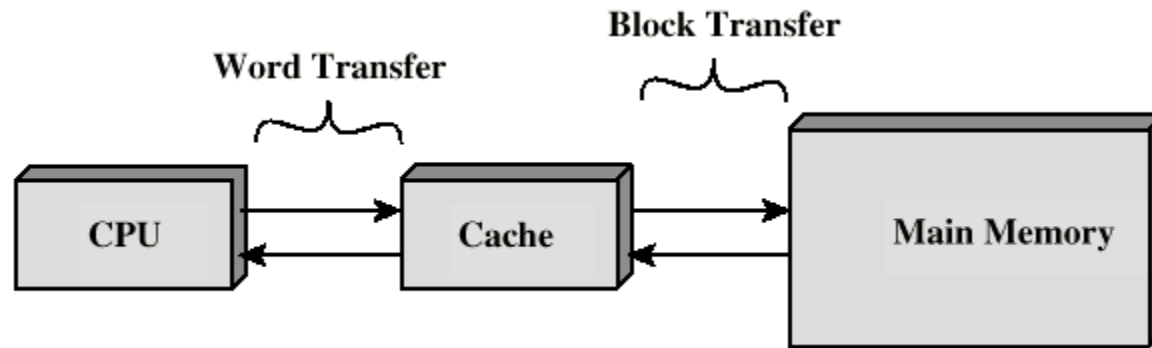


Why Caches Work

- Assume two memories in a system:
 - 1 mS (Memory 1)
 - 100 mS (Memory 2)
- If 95% of memory accesses on faster level, average access time:

$$\begin{aligned}(0.95)(1\text{mS}) + (0.05)(100\text{mS}) &= 0.95\text{mS} + 5\text{mS} \\ &= \mathbf{5.95\text{mS}}\end{aligned}$$

Cache Operation



1. CPU requests memory location.
2. Check cache
3. If present, return from cache ("*cache hit*")
4. If not, read from main memory and copy to cache ("*cache miss*")

Memory Access Time

Calculate *Average Memory Access Time (AMAT)*:

$$AMAT = T_{hit} + MissRate \times MissPenalty$$

T_{hit} = Cache hit time

$MissRate$ = fraction of accesses that miss

$MissPenalty$ = Extra cache miss time (e.g., above T_{hit})

Normally times = CPU Cycles

Data vs. Instruction Caches

If we have an instruction cache (not just data memory), we can convert the above to a *Cycles Per Instruction* (CPI):

$$CPI = CPI_{base} + MemoryAccessesPerInstr \times MissRate \times MissPenalty$$

Why Caches Work (Cont'd)

If we can get the majority of data access to fast memory, we still win big.

Occasional slow copies from memory are fine, worth copying to fast memory when possible.

Why Caches Work: Temporal Locality

```
for(int i=0;i<100;i++)  
    sum += array[0];
```

Memory accessed recently is very likely to be accessed again.

If we access data, very likely we will access it again soon.

Why Caches Work: Spatial Locality

Functions often access data in similar areas (think stack usage!). Or for example another loop:

```
for (int i=0; i<100; i++)  
    sum += array[i];
```

If we access data, very likely we will access nearby data.

Why Caches Work: Instruction Access

Programs tend to follow linear flow.

Many jumps will be within a short area (e.g., think `if()` statements).

Why Caches Work: Compilers & Designers

Compilers (or designers) can optimize both data and instructions for cache usage.

Example of a simple matrix multiply:

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \\ b_4 & b_5 & b_6 \\ b_7 & b_8 & b_9 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix}$$

i-j-k

```
for (i = 0; i < N; i++) {  
    for (j = 0; j < N; j++) {  
        for (k = 0; k < N; k++) {  
            C[i][j] += A[i][k] * B[k][j];  
        }  
    }  
}
```

A[i][k]: walks across a row (good)

B[k][j]: walks down a column (bad)

C[i][j]: same element reused in the inner loop (good)

i-k-j

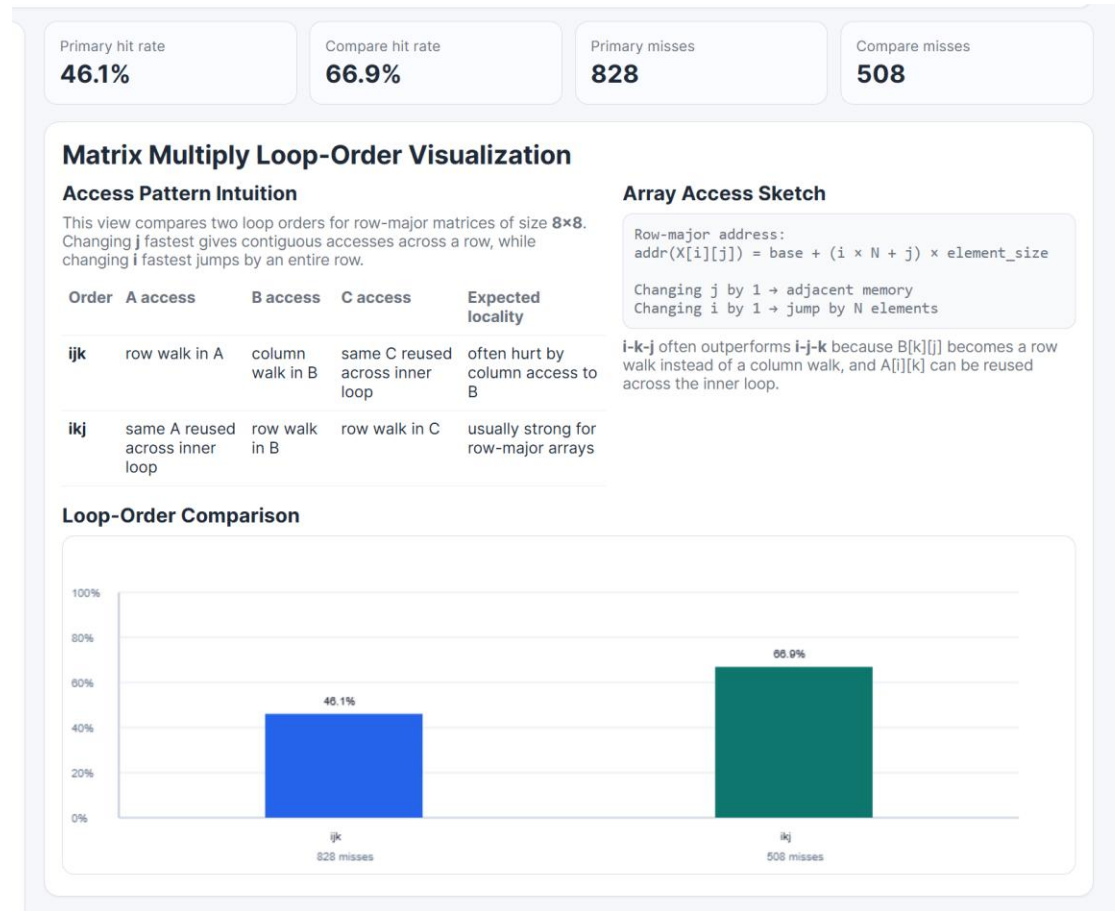
```
for (i = 0; i < N; i++) {  
    for (k = 0; k < N; k++) {  
        for (j = 0; j < N; j++) {  
            C[i][j] += A[i][k] * B[k][j];  
        }  
    }  
}
```

A[i][k]: one value reused across the whole inner (very good)

B[k][j]: walks across a row of B (good)

C[i][j]: walks across a row of C (good)

Demo



(See **cache_simulator_tool.html** in class repo)

Address

`addr(X[i][j]) = base + (i * N + j) * element_size`

- Changing by “j” jumps only by 1 (in cache)
- Changing by “1” jumpers by N (maybe out of cache)

IJK Address Example

0,
1000000,
2000000,
8,
1000080,
2000000,
16,
1000160,
2000000,
24,
1000240,
2000000,
32,
1000320,
2000000,
40,
1000400,
2000000,
48,
1000480,
2000000,
56,
1000560,
2000000,
64,

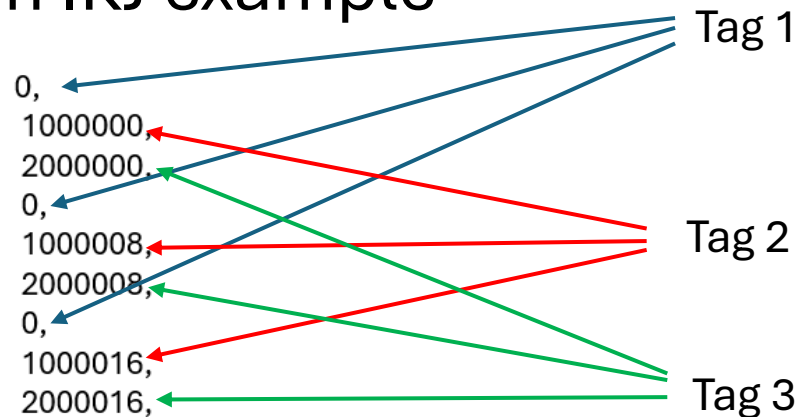
IKJ Address Example

```
0,  
1000000,  
2000000,  
0,  
1000008,  
2000008,  
0,  
1000016,  
2000016,  
0,  
1000024,  
2000024,  
0,  
1000032,  
2000032,  
0,  
1000040,  
2000040,  
0,  
1000048,  
2000048,  
0,  
1000056,  
2000056,  
0,  
,
```

Important Cache Note

- Note that the addresses access different areas of memory.
- The cache is smart enough to track multiple different areas
- It does this through a method of memory tags

e.g., in IKJ example



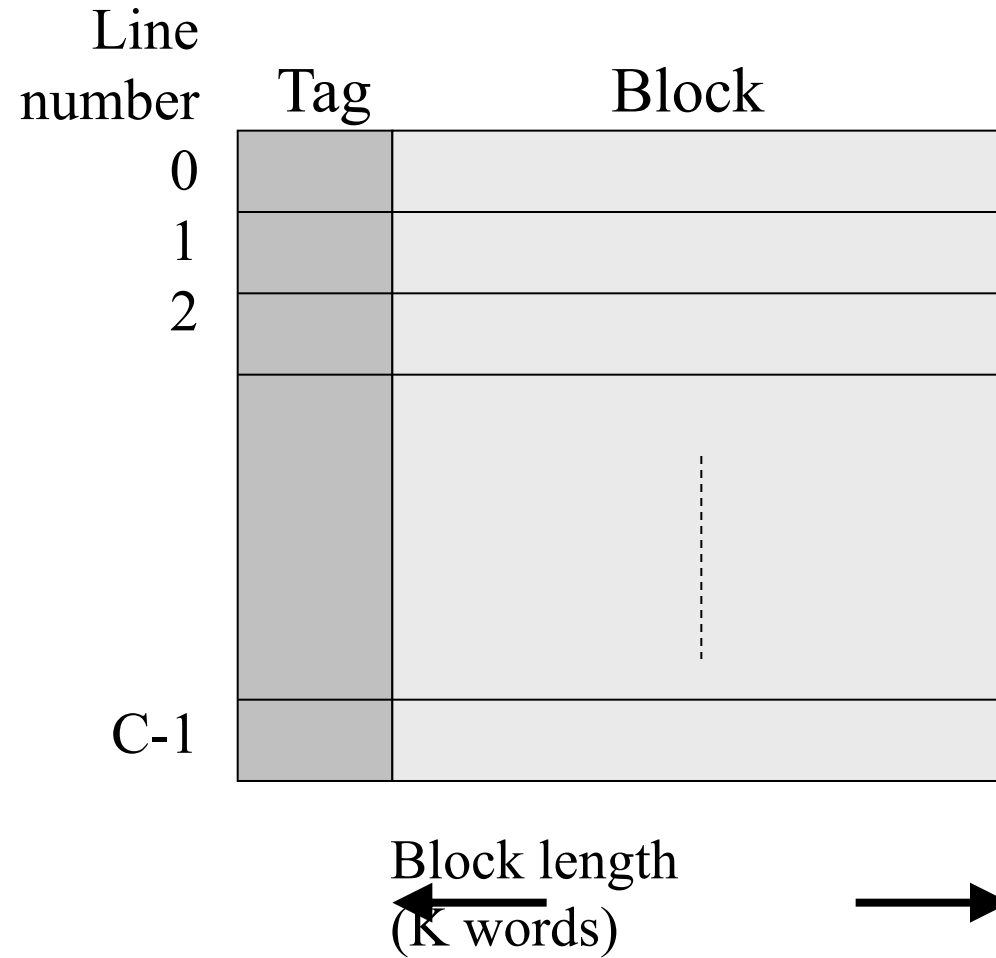
Cache Structure

- Cache includes *tags* to identify the address of a block of main memory.
- Main memory is divided into blocks of K words, meaning have up to:

$$M = 2^n / K \text{ possible blocks}$$

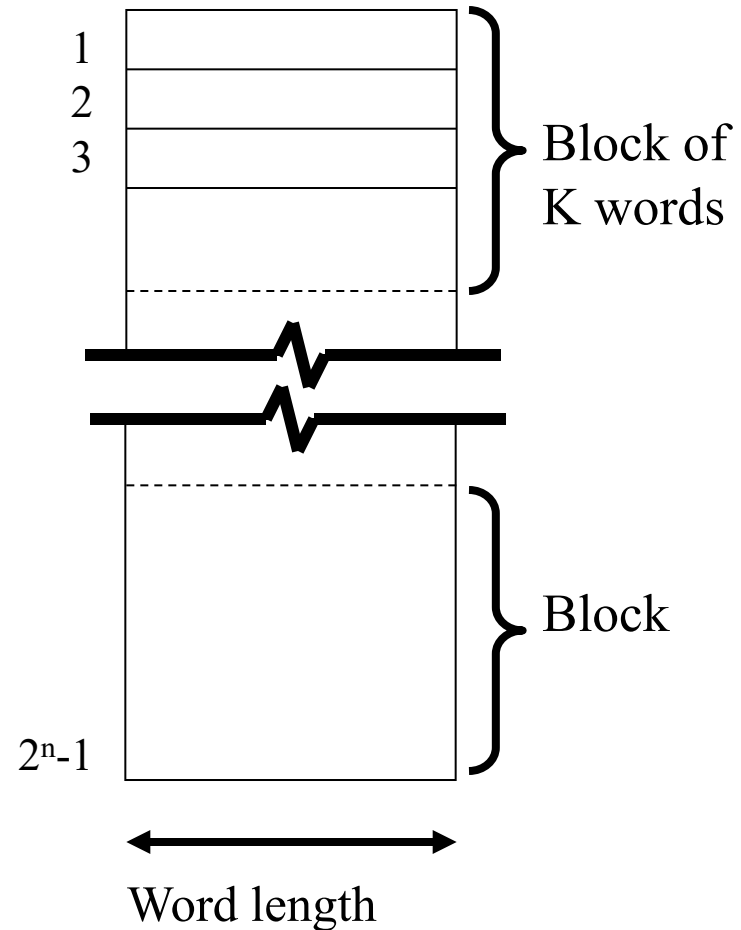
- Cache could have up to C blocks of main memory

Cache Structure (continued)



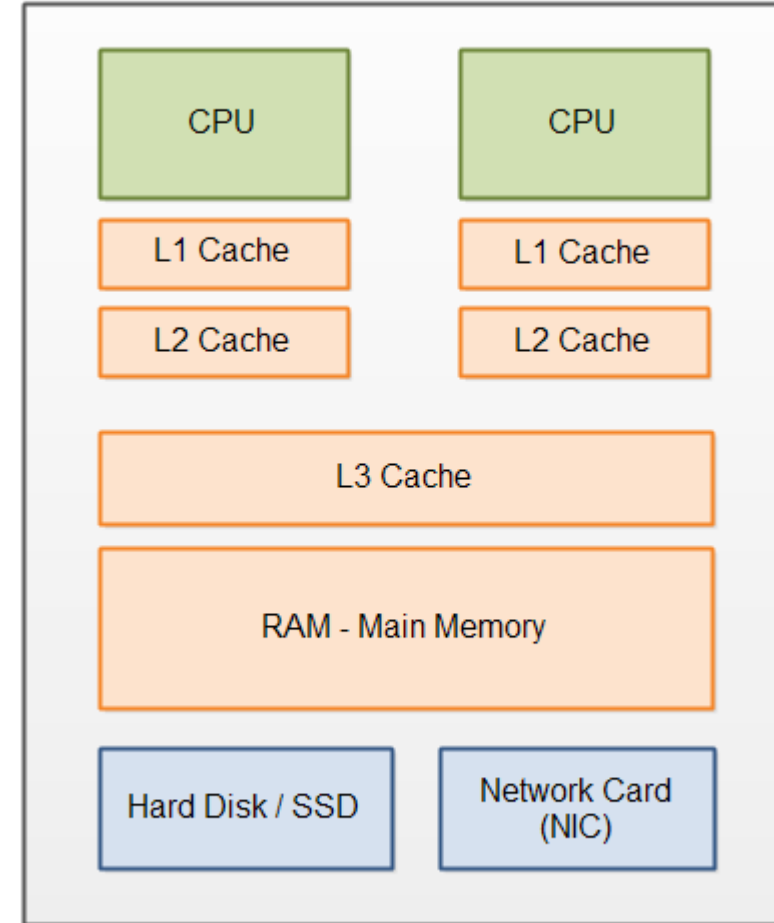
Memory Divided into Blocks

Memory
Address



Memory Hierarchy with Multiple Caches

- Modern CPUs may include multiple levels of caches – L3 / L2 / L1
- Small embedded devices typically have one level but may have different instruction & data caches



Multi-Level Access Time

$$AMAT = T_{L1} + MR_{L1}(T_{L2} + MR_{L2} \times MissPenalty_{Mem})$$

T_{L1} = Level 1 hit time

MR_{L1} = Miss Rate on Level 1

T_{L2} = Level 2 hit time

MR_{L2} = Miss Rate on Level 2

$MissPenalty_{Mem}$ = Extra cache miss time (e.g., above MR_{L2})

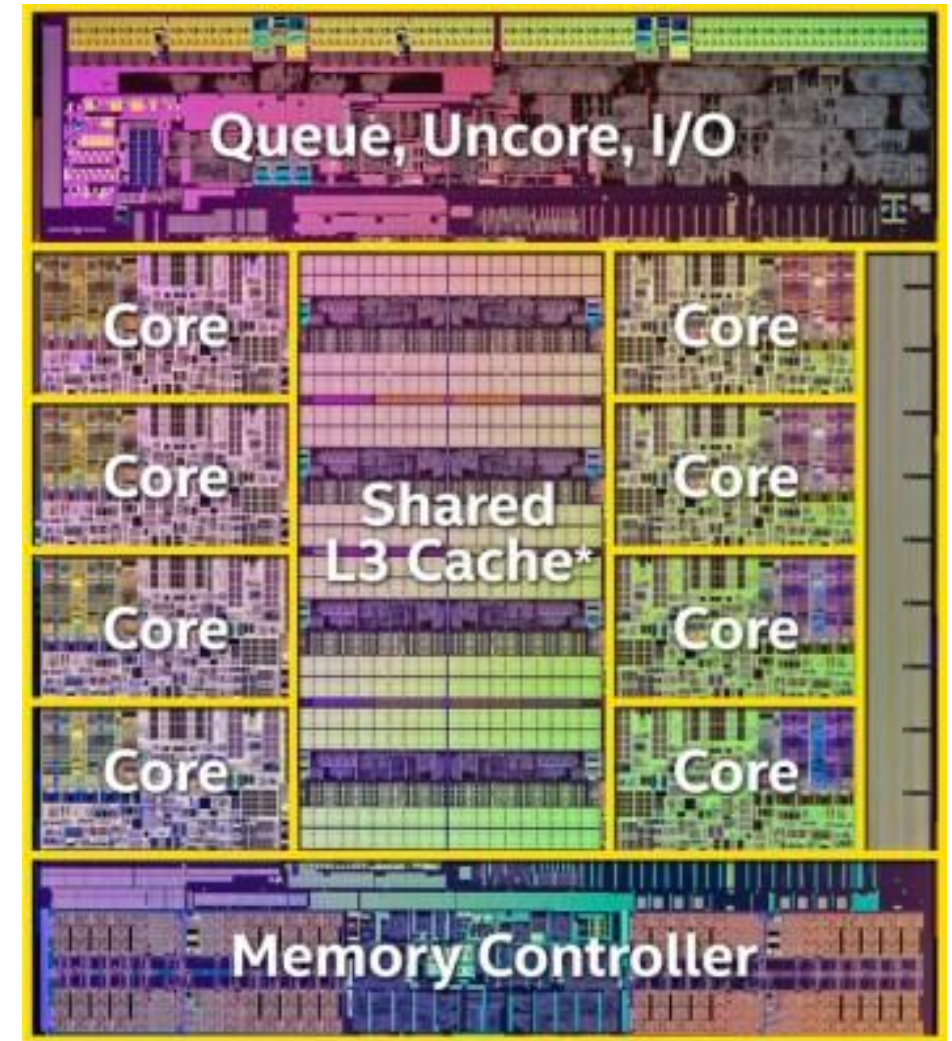
Normally times = CPU Cycles

Formula Intuition

- Multiple levels of caches allow us to optimize by going for:
 - Very fast L1 that has a moderate miss rate
 - “Moderate miss rate” → Smaller
 - Fast L2 with a lower miss rate
 - “Lower miss rate” → Larger

Memory Hierarchy

- Compare the area of the L3 cache to the CPU core itself (L2 & L1 are inside each of the cores)
- Memory access is one of the most fundamental problems in CPU speed.



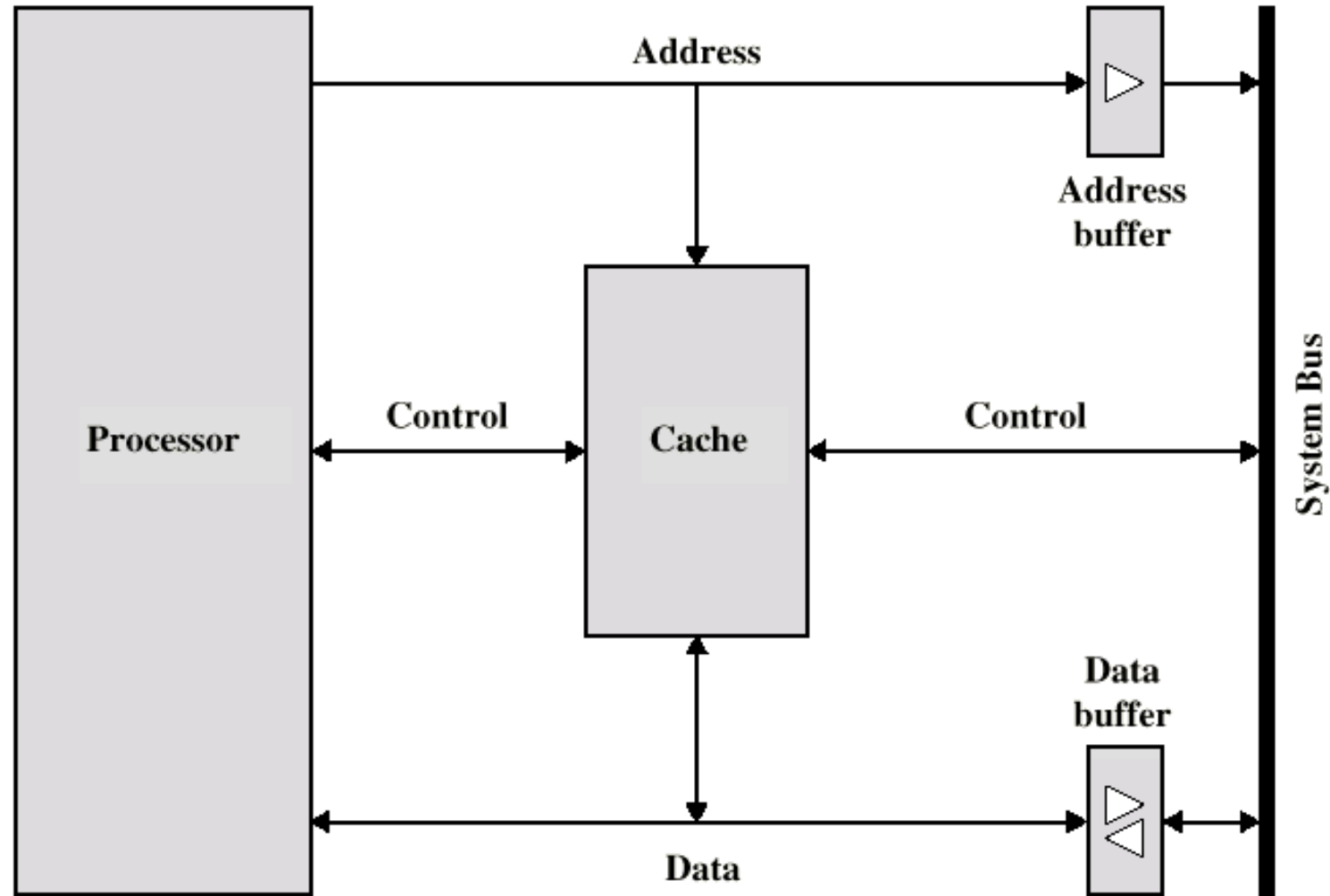
Cache Design Questions

- Size
- Mapping Function
- Replacement Algorithm
- Write Policy
- Block Size
- Number of Caches

Cache size

- Cost – More cache is expensive
- Speed
 - More cache is faster (up to a point)
 - Larger decoding circuits slow up a cache
 - Algorithm is needed for mapping main memory addresses to lines in the cache. This takes more time than just a direct RAM

Typical Cache Organization



Mapping Functions

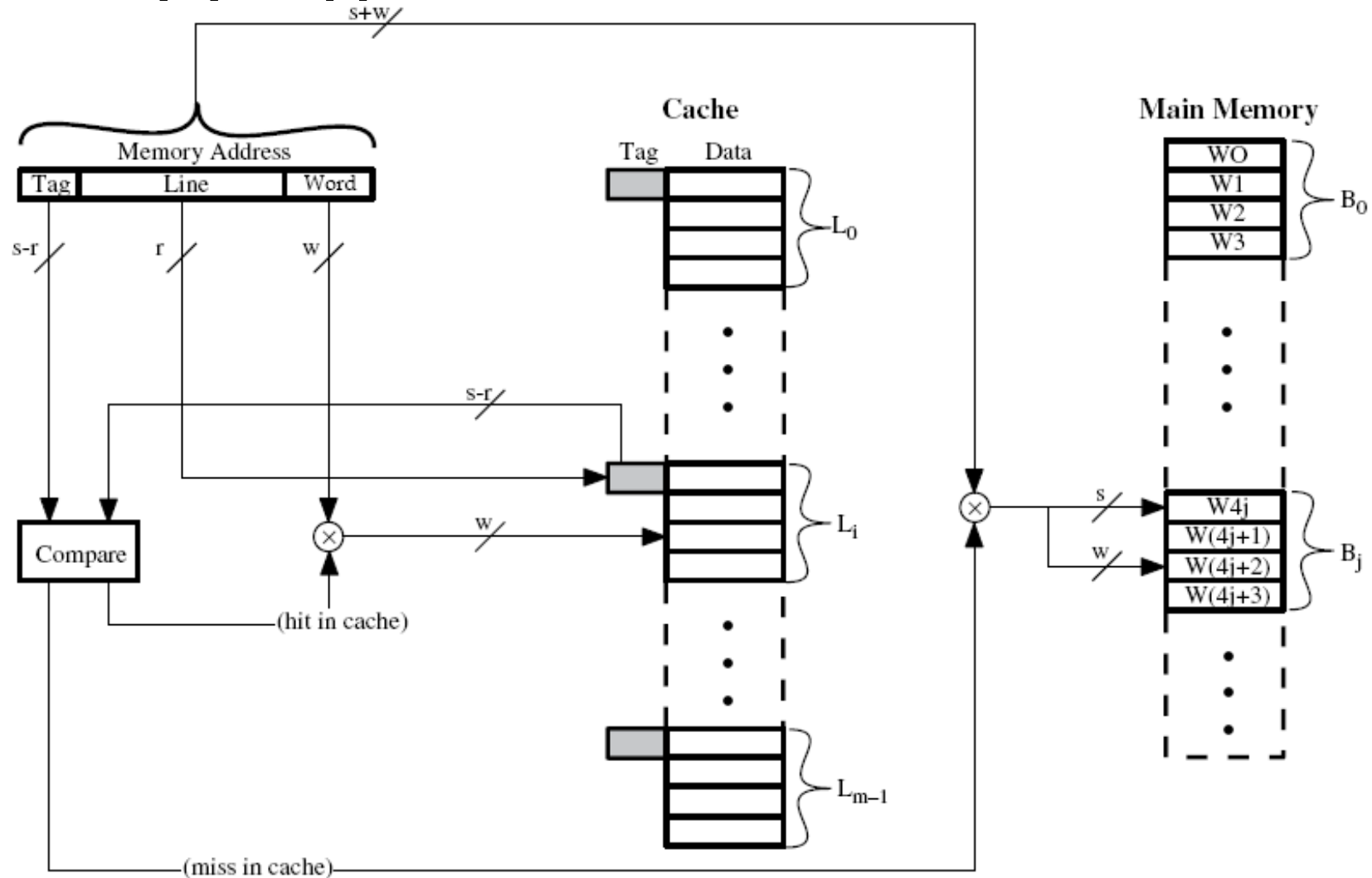
- A mapping function is the method used to locate a memory address within a cache
- It is used when copying a block from main memory to the cache and it is used again when trying to retrieve data from the cache
- There are three kinds of mapping functions
 - Direct
 - Associative
 - Set Associative

Direct Mapping

- Each block of main memory maps to only one cache line
 - If a block is in memory, it is always found in the same cache line

Cache line	Main Memory blocks held
0	0, m, 2m, 3m... $2^s - m$
1	1, m+1, 2m+1... $2^s - m + 1$
m-1	m-1, 2m-1, 3m-1... $2^s - 1$

Direct Mapping Access



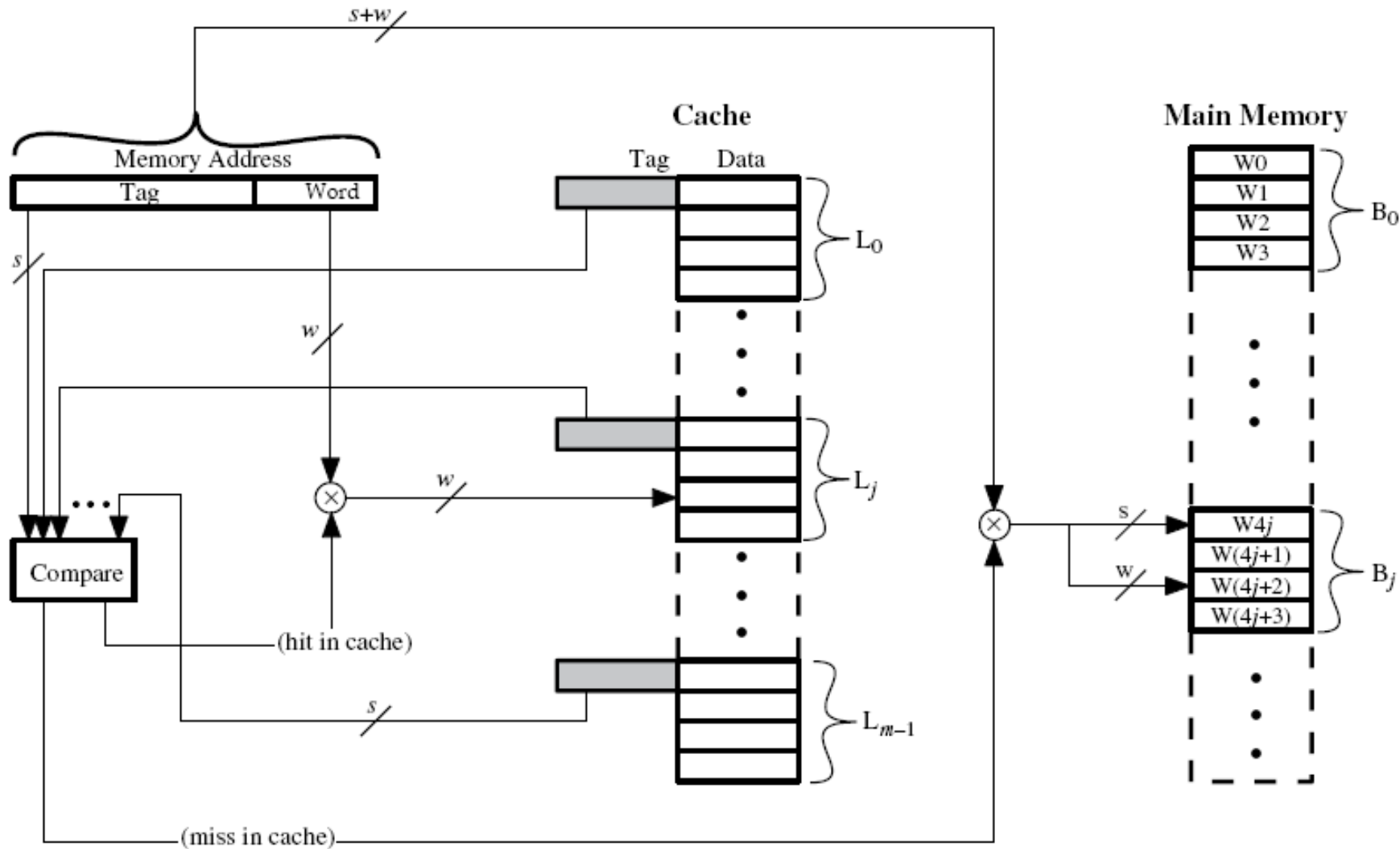
Direct Mapping Features / Problems

- Simple & Easy / Fast to Implement
 - Can calculate cache line from address
- A program accessing 2 blocks mapping to the same line repeatedly causes “thrashing”
 - This has very poor performance as cache has constant misses

Associative Mapping

- Load blocks as used into cache lines
- Need to search cache lines to find possible memory

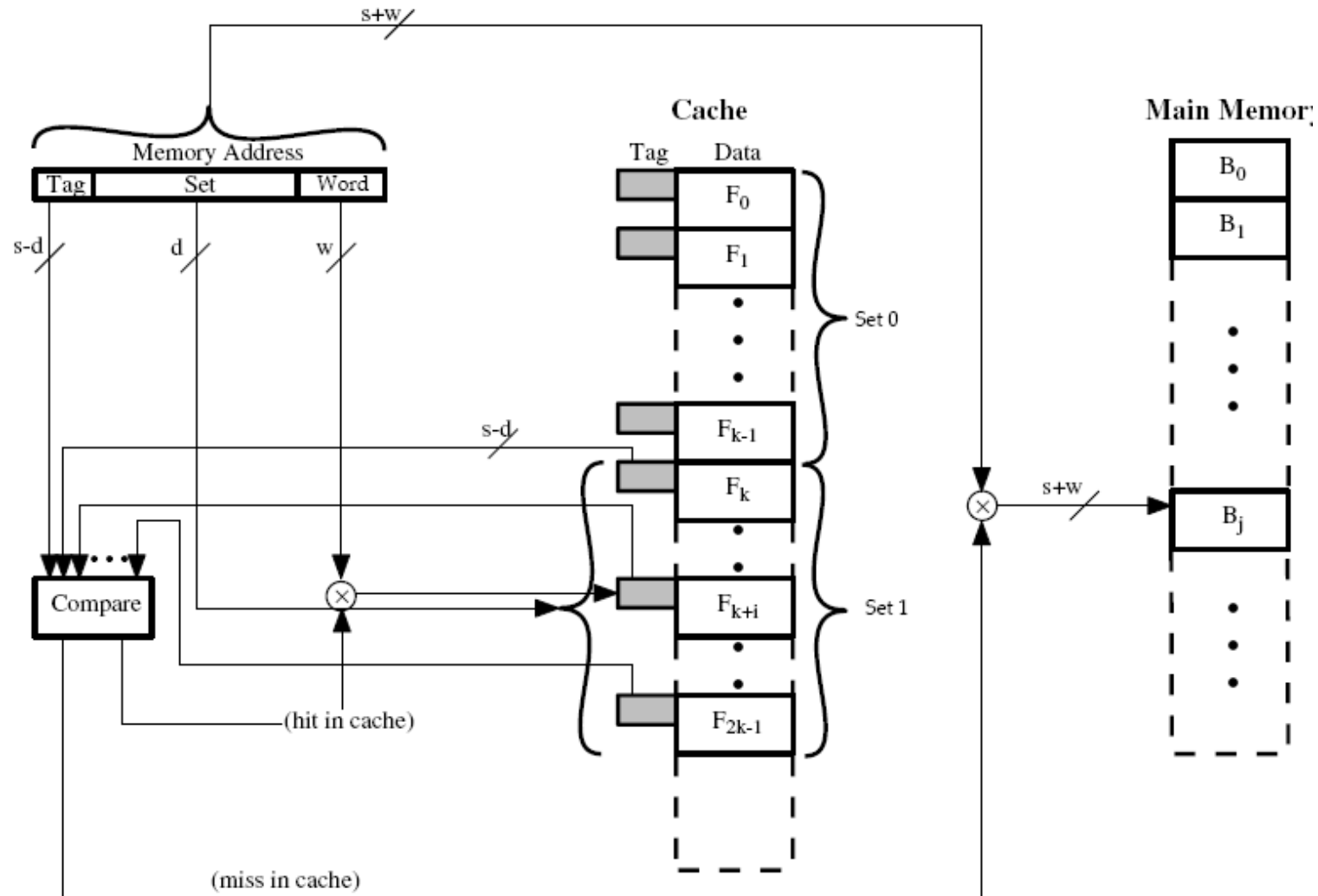
Fully Associative Mapping



Fully Associative Mapping Problems

- Search of a large cache is problematic! Ruins the point of a cache...
- So split the search up into smaller sections.
- We say there is k blocks/lines in each set
 - $k = 1$ is direct mapping (only one block it can be in)
 - $k = 2$ is common (2 possible blocks to search)

K-Way Associative Mapping



Replacement Algorithm

- For direct mapping no algorithm – we have only one choice for where a new block goes.
- Associative & Set Associative have choices. Typically:
 - Least Recently Used (LRU)
 - Replace block not touched in the longest time
 - With 2-way associative store this in a bit, set a bit when one set is used. Replace unused set.
 - First In First Out (FIFO) needed for more sets

Writing to Cache

- What happens when your processor writes to memory?
- Complexities:
 - If we write to cache, main memory is invalid.
 - If we write to memory, cache is invalid.
 - Consider also multiple CPU cores, or peripherals (DMA) accessing memory.

Write through

- All writes go to main memory and cache
- Other CPUs monitor main memory traffic to keep cache up-to-date
- Downsides:
 - Lots of bus traffic!
 - Slows down writes (memory access on every write)

Write back

- Writes update cache only
- Cache tracks that an update (write) has happened
- If block in cache to be removed (flushed), check if update happened, if so *write back* to main memory
- Downsides:
 - Other caches get out of sync
 - I/O must access main memory through cache

Dealing with Multi-core

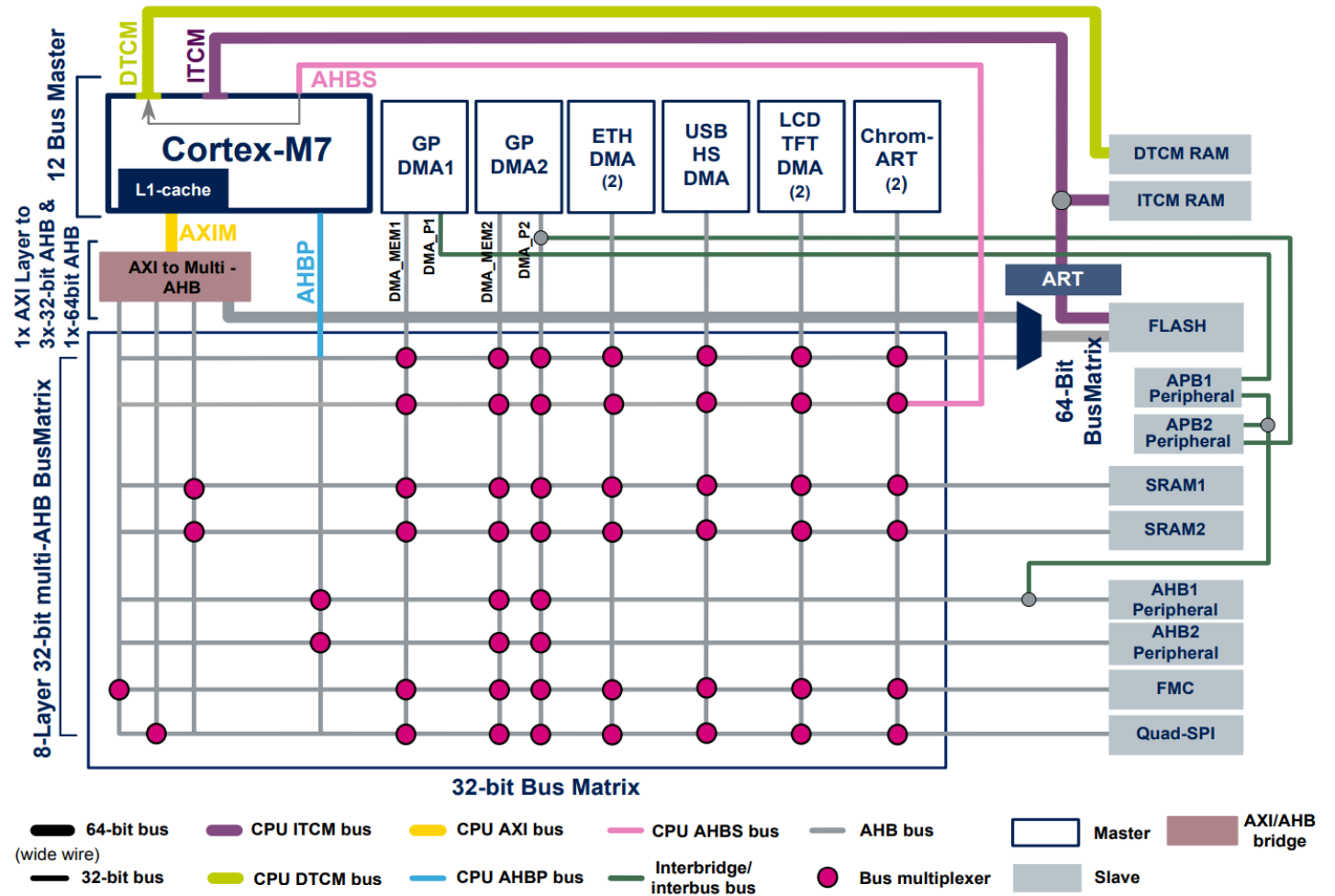
- Processors monitor bus to see if any writes of interest to them have occurred
- Overseer monitors all caches and forces synchronization between them
- Memory is marked as non-cacheable

Embedded Processor Caches

- Smaller embedded systems will often not have caches.
- Normally there is a separate instruction cache (ICACHE) and data cache (DCACHE)
- STM32F303 (Cortex-M4 based) does not have either.

e.g., STM32F7

Figure 1. STM32F7 Series system architecture

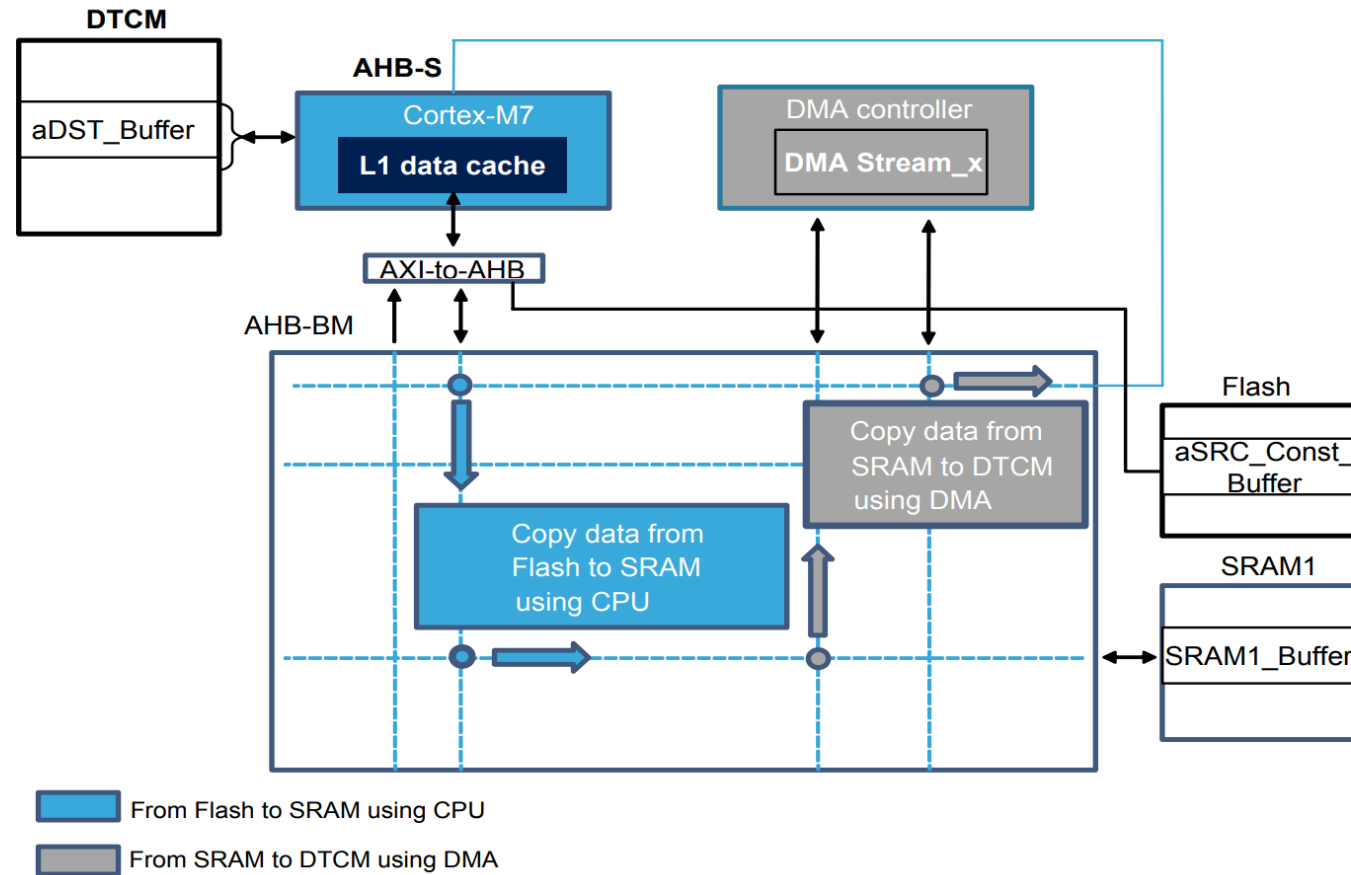


e.g., Default Cache Setup

Table 2. Memory region shareability and cache policies

Address range	Memory region	Memory type	Shareability	Cache policy
0x00000000-0x1FFFFFFF	Code	Normal	Non-shareable	WT
0x20000000-0x3FFFFFFF	SRAM	Normal	Non-shareable	WBWA
0x40000000-0x5FFFFFFF	Peripheral	Device	Non-shareable	-
0x60000000-0x7FFFFFFF	External RAM	Normal	Non-shareable	WBWA
0x80000000-0x9FFFFFFF				WT
0xA0000000-0xBFFFFFFF	External Device	Device	Shareable	-
0xC0000000-0xDFFFFFFF			Non-shareable	
0xE0000000-0xE0FFFFFF	Private peripheral bus	Strongly-ordered	Non-shareable	-
0xE0100000-0xFFFFFFFF	Vendor-specific system	Device	Non-shareable	-

e.g., Cache Problems



e.g., Fixing Data Coherency

- Solution 1:
Manual cache maintenance operation: SCB_CleanDCache() when coherence problem exists (DMA).
- Solution 2:
Modify SRAM1 to be “Write-Through” policy (WT instead of WBWA)
- Solution 3:
Modify SRAM1 to be shared attribute (prevents D-cache usage)
- Solution 4:
Modify all writes to be “Write-Through”

Memory Caches & Performance

- Caching is an important solution to the memory wall problem.
- Caching also helps with general bus contention problems.
- Different cache access patterns can drastically affect software performance.
- Reviewed cache tags & replacement policy.
- Reviewed some examples of cache usage on STM32F7